

Сервис rights-cache

Участок: Асинхронный сервис кэширования (rights-cache)

У существующего сервиса кэширования методов бизнес-логики есть особенность, которая затрудняет его использование в системе прав. При наступлении изменений данные в сервисе кэширования немедленно инвалидируются, после этого при первом вызове метода чтения происходит расчет актуальных данных и запись их в сервис кэширования. Это позволяет всегда возвращать пользователю актуальные данные, но вносит задержку при первом чтении после инвалидации.

Для системы прав подобные задержки неприемлемы, но зато считается нормальным, когда изменения в правах применяются не моментально.

Для решения этой проблемы создан специализированный сервис кэширования с асинхронной инвалидацией. Сервис оформлен в виде шаблона `async-cache` (расположен в репозитории [sbis/middleware](#)), чтобы была возможность применить аналогичный сервис для каких-то других задач. Для системы прав создан экземпляр сервиса с названием `rights-cache`.

Ключевые возможности

- **Key-value семантика**

Key - строка, ограниченная 1000 символов. Value - произвольные бинарные данные. В качестве хранилища используется Redis.

- **Отсутствие инвалидации**

Если данные "устарели", то они продолжают возвращаться еще какое-то время.

- **Очередь на асинхронный пересчет**

Очередь организуется на основе модуля [Queue](#).

Организация очереди

Очередь организована таким образом, чтобы не создавать большую нагрузку на отдельный юнит online при массовой инвалидации данных, например, в результате какого-то скрипта. Кроме того, "схлопываются" задачи на пересчет, если по какому-то объекту данные меняются несколько раз за короткий промежуток времени.

Задача "схлопывания" решена следующим образом. При первом вызове `AsyncUpdate` добавляем в очередь задачу на пересчет, а также делаем в Redis пометку, что задача на пересчет уже есть. Последующие вызовы `AsyncUpdate` сначала проверяют пометку и при ее обнаружении ничего не делают. Метку снимаем при разборе очереди, но до фактического обновления данных, чтобы не пропустить изменение данных в процессе пересчета.

Для ограничения нагрузки:

- в качестве ключа для очереди используется идентификатор группы серверов online, благодаря этому при разборе очереди все задачи от одной группы попадают в одну пачку.
- сделано достаточно большое число секций очереди (на момент выпуска сервиса используется 10 секций), т.е. в одну секцию попадают данные от нескольких случайных групп серверов. Это позволяет минимизировать риски, когда из-за проблем на одной группе серверов, не обновляются данные на другой.

Клиент rights-cache

Для работы с сервисом `rights-cache` создан модуль "Async Cache Client" (расположен в репозитории [sbis/service-engine](#)), который предоставляет нижеуказанные методы.

Тут в качестве `shard_name` ожидается название экземпляра `async-cache`, т.е. в случае системы прав это всегда `rights-cache`.

ByteVector Read(String shard_name, String key, String read_method_name)

- читает данные с `rights-cache` по ключу, если данные есть, то они сразу возвращаются.

- через Distributed Lock накладывает блокировку на key
- снова проверяет наличие данных на rights-cache, возвращает их при наличии
- при отсутствии данных вызывает read_method_name для получения новых данных
- сохраняет данные на rights-cache и возвращает их вызывающему коду

void Write(String shard_name, String key, String read_method_name)

- через Distributed Lock пытается наложить блокировку на key, если не удалось, то значит расчет уже идет и больше ничего не делается.
- вызывает read_method_name для получения новых данных
- сохраняет данные на rights-cache

void AsyncUpdate(String shard_name, String key, String update_method_name)

- безусловно вызывает AsyncUpdate на rights-cache

API сервиса rights-cache

AsyncCache.Read - прочитать данные по ключу. Если данных нет, то возвращается null.

AsyncCache.Write - записать данные по ключу.

AsyncCache.AsyncUpdate - добавить в очередь задачу на пересчет данных по ключу.

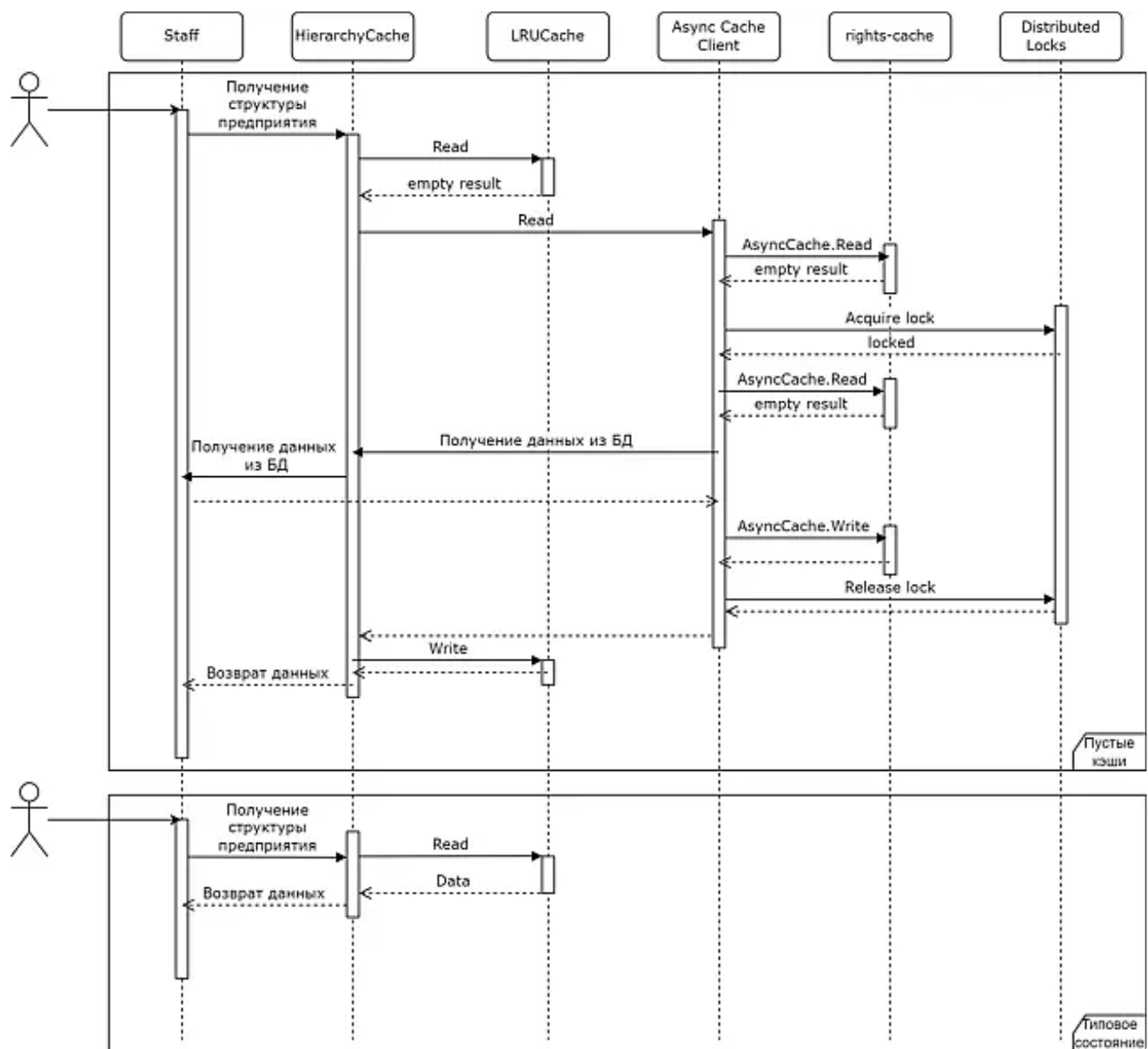


Рис. Схема взаимодействия прикладного кода с HierarchyCache и rights-cache